Encrypted IoT Device Fingerprinting & Anti-Spoofing via Traffic Timing Analysis

A Complete Technical Guide and Project Treatise

Guide Overview: How to Read This Project

If you are reviewing, presenting, or preparing for an examination on this project, here is the core mental model:

1. **The Fundamental Problem:** Smart home devices transmit sensitive data across networks, but their traffic is encrypted. You cannot see what they are saying (no payload inspection). Worse, their identity labels (MAC addresses, IP addresses) are trivial for an attacker to spoof.
2. **The Scientific Solution:** You don't need to read the encrypted payload to know who is talking. Every device has unique physical hardware (crystal oscillators, processor clocks, firmware execution loops, and sensor polling schedules) that causes it to transmit packets with distinct **timing rhythms**. We measure the time between consecutive packets (**Inter-Arrival Time, or IAT**).
3. **The Engineering Achievement:**

• We extract 13 statistical timing moments from packet arrival streams.

• A trained Random Forest model identifies authorized devices with **99.34% accuracy**.

• An open-set rejection threshold ($\tau = 0.84$) detects unauthorized foreign devices with a **95.66% rejection rate**.

• A 3-state claimed-identity verification engine detects MAC/IP impersonation attacks with a **100.0% detection rate** (0% bypass).

• Everything is deployed as a live full-stack web dashboard (FastAPI + React Vite).

Chapter 1: The Core Intuition — Why Packet Timing Works

1.1 The Vulnerability of Modern IoT Networks

Smart home and industrial IoT deployments are plagued by two opposing security challenges:

- **Payload Encryption (The Blind Spot):** Modern standards mandate TLS/HTTPS, SSH, or DTLS encryption. Traditional firewalls and Deep Packet Inspection (DPI) engines inspect packet bytes to identify malware or verify endpoints. When encryption is enabled, DPI becomes useless unless network operators deploy complex, privacy-violating MITM proxy certificates.
- **Trivial Identifier Forgery (The Identity Crisis):** Network routers determine which device is communicating by reading the Layer-2 MAC address and Layer-3 IP address. These headers are completely unauthenticated plaintext. An attacker using a cheap Raspberry Pi or laptop can spoof an authorized smart security camera's MAC address in seconds using a single command:

`bash

sudo ip link set dev eth0 address 00:16:6c:ab:6b:88

`

Once spoofed, the router blindly trusts the rogue device, granting it full network access.

1.2 The Physics of Hardware Timing: Digital Fingerprints

Even if an attacker clones the MAC address, IP address, and protocol headers of an authorized smart camera, **they cannot easily clone the physical dynamics of the camera's silicon and firmware**.

Consider how an IoT device operates:

1. **Crystal Oscillators and Hardware Clock Drift:** Every circuit board contains a physical quartz crystal oscillator that ticks to drive the CPU clock. Due to microscopic manufacturing tolerances, temperature variations, and supply voltage fluctuations, each hardware chip exhibits tiny, characteristic timing drift.
2. **RTOS Firmware Task Scheduling:** IoT microcontrollers (e.g., Espressif ESP8266, ARM Cortex-M, MIPS SoC) run lightweight Real-Time Operating Systems (FreeRTOS, Zephyr) or bare-metal loops. The internal scheduler wakes up, polls sensors, handles network interrupts, and queues packets on rigid, predictable microsecond intervals.
3. **Peripheral Bus Latencies:** A PIR motion sensor requires specific analog stabilization time; a video compression chip encodes video frames in cyclical bursts; a smart plug polls an analog-to-digital power meter every few milliseconds.

These combined hardware and software characteristics impart a distinct **timing signature** to packet emissions. Even though the payload is encrypted, the **gaps between packets** are exposed in plaintext.

```
Device A (Amazon Echo): Packet 1 ■■■■■■■[ 45 ms ]■■■■■■■> Packet 2 ■■■■■■■[ 46 ms ]■■■■■■■> Packet 3 ■■■■■■■[ 44 ms
]■■■■■■■> Packet 4 Device B (TP-Link Plug): Packet 1 ■■■■■■■■■■■■■■■■■■■■■■■■■■[ 240,000 ms (Sleep)
]■■■■■■■■■■■■■■■■■■■■■■■■■■> Packet 2 ■■[ 2 ms ]■■> Packet 3
```

By analyzing these inter-packet intervals, we can recognize authorized devices, reject unknown hardware, and immediately catch an attacker attempting to impersonate an authorized device.

Chapter 2: The Dataset & Preprocessing Pipeline

2.1 The UNSW Sydney Smart Home Dataset

To ensure our findings are scientifically valid and reproducible, we use the public **UNSW Sydney Smart Home Benchmark Dataset** (captured by Sivanathan et al., IEEE Transactions on Mobile Computing).

- **Capture Duration Used:** Exactly 72 continuous hours (November 22, 23, and 24, 2016) [configs/config.yaml].
- **Real Network Environment:** Traffic was collected from a real university smart-home living lab containing dozens of real consumer IoT products connected over Wi-Fi and Ethernet.

2.2 The 5 Enrolled Known Devices

We chose 5 distinct smart-home appliances spanning different manufacturers, hardware platforms, and network traffic behaviors:

1. **Amazon Echo (Smart Speaker):** High interactive traffic; frequent audio telemetry bursts, DNS queries, and consistent NTP/keepalive beaconing.
2. **Belkin WeMo Motion Sensor (PIR Security Sensor):** Event-driven; sends periodic XML/UPnP status updates, punctuated by rapid bursts when physical motion is detected.
3. **Philips Hue Bridge (Lighting Controller):** Continuous local ZigBee bridge polling, periodic cloud synchronization heartbeats.
4. **Samsung SmartCam (Wi-Fi Camera):** Continuous high-rate video frame streaming, yielding rapid packet trains with small inter-arrival times.
5. **TP-Link Smart Plug (Smart Energy Meter):** Extreme low-rate traffic; sleeps for ~4 minutes at a time, waking briefly to send a short telemetry burst before sleeping again.

Enrolled Device Class	MAC Address	Total Packets Captured	128-Packet Windows ($W=128$)	Proportion of Dataset
AmazonEcho	`44:65:0d:56:cc:d3`	175,410	1,370	18.0%
BelkinWemoMotionSensor	`ec:1a:59:83:28:11`	376,601	2,942	38.7%
PhilipsHue	`00:17:88:24:76:ff`	229,885	1,795	23.6%
SamsungCamera	`00:16:6c:ab:6b:88`	183,539	1,433	18.8%
TPLinkSmartPlug	`50:c7:bf:00:56:39`	8,544	66	0.9%
Total Enrolled	—	**973,979**	**7,606**	**100.0%**

(Source: `data/processed/*.csv`, `data/features/*_windows.csv`)

2.3 The 22 Unseen / Foreign Devices in the Catalog

A major strength of this project is testing what happens when **unauthorized devices** appear on the network:

- The UNSW benchmark catalog contains 22 other devices.
- **7 Devices were Offline:** `BlipCareBPMeter`, `HelloBarbie`, `RingDoorBell`, `TPLinkCamera`, `WithingsBabyMonitor`, `BelkinCamera`, and `AugustDoorBell` had 0 packets recorded during Nov 22–24 (they were inactive in the testbed) [results/unknown_devices_summary.csv].
- **15 Devices were Active:** We evaluated 4,987 windows across all 15 active devices (e.g., Nest Dropcam, HP Printer, Netatmo Weather, iHome, LiFX Bulb, Canary Camera) to test our open-set defense [results/unknown_evaluation_table.csv].

2.4 Preventing Data Leakage: Temporal Block Splitting

A common beginner mistake in machine learning is randomly shuffling time-series packets into train and test sets.

- **Why Random Shuffling Fails:** If Packet 100 is in the training set and Packet 101 is in the test set, the model simply memorizes the local conversation session. It achieves an artificial 99.9% accuracy in the lab, but fails completely when deployed next week.
- **Our Solution (Temporal 1-Hour Blocks):** Continuous traffic is chopped into 1-hour time blocks [src/splits.py].
- The first **70% of chronological hours** are assigned to the **Training Split** (5,468 windows).
- The next **15% of chronological hours** form the **Validation Split** (1,071 windows) — used strictly for threshold tuning.
- The final **15% of chronological hours** form the untouched **Test Split** (1,067 windows) — touched only once for the final exam.
- **Scaler Isolation:** The `StandardScaler` is fitted **strictly on the training split**, then frozen. It scales validation and test data using training statistics, completely preventing look-ahead leakage.

Chapter 3: Step-by-Step Feature Engineering Pipeline

Here is how raw network packets are transformed into a mathematical fingerprint:

```
[Raw Network Traffic] ■ ▼ (Step 2: Preprocessing) [Strip MAC, IP, Payload → Keep only chronological timestamps t_i]
■ ▼ (Step 3: IAT Calculation) [Compute IAT = t_i - t_{i-1} in seconds] ■ ▼ (Step 4: Windowing & Normalization)
[Group into W=128 consecutive IATs, apply log1p: z = ln(1 + IAT)] ■ ▼ (Step 5: Feature Extraction) [Compute 13
Summary Statistical Moments per window] ■ ▼ (Step 6: Scaling) [Normalize with Training Scaler: (z - mean_train) /
std_train] ■ ▼ (Step 7: Classification & Decision Engine) [Random Forest Inference + Rejection Threshold tau=0.84]
```

3.1 Step 2: Sanitization & Identifier Stripping (`src/preprocessing.py`)

All packet payloads, MAC addresses, IP addresses, port numbers, TCP window sizes, and protocol flags are permanently discarded. The output contains only:

- Arrival timestamp (epoch time with microsecond resolution).
- Packet size in bytes.

3.2 Step 3: Computing Inter-Arrival Times (`src/iat.py`)

For every consecutive pair of packets from a device:

$$\text{IAT}_i = t_i - t_{i-1}$$

If clock drift causes a microscopic backward step ($\Delta t < 0$), it is clamped to 0.0 to guarantee numerical stability.

3.3 Step 4: Non-Overlapping Windowing & The Log Transform (`src/windowing.py`)

- **Window Size ($W=128$):** Packets are grouped into consecutive non-overlapping windows of 128 IAT values (spanning 129 physical packets). Trailing fragments with fewer than 128 packets are discarded without artificial padding.
- **The Log Transform ($\log(1 + \text{IAT})$):** IoT packet delays vary wildly. A sensor might send two packets separated by 0.001 seconds, and then sleep for 300 seconds. If you feed raw numbers $[0.001, \dots, 300.0]$ into statistical formulas, the single 300-second number completely dominates the mean and variance.

Applying $z = \ln(1 + \text{IAT})$ compresses this scale smoothly:

- $\ln(1 + 0.001) \approx 0.001$ (sub-millisecond bursts stay small).
- $\ln(1 + 300.0) \approx 5.70$ (large sleep times are pulled into a well-behaved range).

3.4 Step 5: The 13 Statistical Moments (`src/features.py`)

Instead of feeding 128 raw numbers into a model, we compute 13 compact statistical summary metrics that capture the shape of the timing distribution:

1. `raw_mean` **(μ):** The average packet pace. Fast-streaming cameras have very small means; idle plugs have large means.
2. `raw_std` **(σ):** The timing variance. Devices with regular clock ticks have low standard deviation; bursty devices have high standard deviation.
3. `raw_min`**:** The fastest possible back-to-back packet transmission speed of the device's network card.
4. `raw_max`**:** The longest sleep or idle timeout interval.
5. `raw_median` **(50th Percentile):** The robust center of traffic pacing, immune to occasional outliers.
6. `raw_skew`**:** Asymmetry of timing. Indicates whether packet delays cluster near zero with rare long delays (positive skewness).
7. `raw_kurtosis`**:** Tail heaviness / peakedness. Distinguishes Gaussian jitter from heavy-tailed, sporadic event traffic.
8. `raw_p25` **(25th Percentile):** Pacing of rapid burst exchanges.
9. `raw_p75` **(75th Percentile):** Transition boundary toward idle state. *(Most important feature in our model!)*
10. `raw_p95`**:** 95th percentile pacing.
11. `raw_p99`**:** 99th percentile pacing (near-maximum sleep boundaries).
12. `raw_energy`**:** $\sum z_i^2$ — the total signal power of the timing sequence.
13. `raw_entropy`**:** Shannon spectral entropy. Measures how unpredictable or random the packet transmission timing is.

Chapter 4: The Closed-Set Classifier Results

4.1 Why Random Forest?

We chose a Random Forest ensemble (100 decision trees) [configs/config.yaml].

- **Sub-millisecond Speed:** Tree traversal requires only a few simple `if/else` comparisons. A window is classified in less than 2 milliseconds, making it suitable for low-power edge routers.
- **Outlier Immunity:** Individual anomalous network stalls do not corrupt tree threshold boundaries.
- **Probability Outputs:** Random Forest outputs well-calibrated class probability vectors $\mathbf{P} = [P_1, P_2, \dots, P_K]$ by averaging votes across all 100 trees, which is essential for confidence-based anomaly rejection.

4.2 Benchmark Results on the Test Set (1,067 Windows)

The model was trained on 5,468 training windows and evaluated on the untouched 1,067 test windows [results/step7_20261026_151020/results_summary.json]:

- **Test Accuracy: 99.34%** (1,060 out of 1,067 windows classified correctly).
- **Test Macro F1-Score: 99.38%**.

Device Class	Precision	Recall	F1-Score	Number of Test Windows
AmazonEcho	100.0%	97.55%	98.76%	204

BelkinWemoMotionSensor	99.76%	99.76%	99.76%	423
PhilipsHue	98.37%	100.0%	99.18%	242
SamsungCamera	98.95%	99.47%	99.21%	189
TPLinkSmartPlug	100.0%	100.0%	100.0%	9

4.3 Confusion Matrix Analysis

Out of 1,067 test windows, there were **only 7 misclassifications in total**:

- 5 windows of Amazon Echo were mistaken for Belkin (1), Hue (2), or Camera (2).
- 1 window of Belkin Motion was mistaken for Hue.
- 1 window of Samsung Camera was mistaken for Hue.
- TP-Link Smart Plug had zero errors (9/9 correct).

4.4 Feature Importance Hierarchy (What the Model Cares About Most)

Analyzing Gini impurity reduction across all 100 trees reveals which features carry the strongest fingerprinting signal:

1. `raw_p75` (75th Percentile): **16.7%** importance.
2. `raw_median` (50th Percentile): **13.6%** importance.
3. `raw_p25` (25th Percentile): **13.3%** importance.
4. `raw_min` (Fastest packet turnaround): **11.3%** importance.
5. `raw_kurtosis` (Tail heaviness): **11.3%** importance.

Intuition: The model relies heavily on the **inter-quartile pacing** (`p25`, `median`, `p75`) and the **fastest possible hardware response time** (`min`), because these reflect the physical CPU clock and firmware processing loop.

Chapter 5: Solving the Open-World Problem (Unknown Device Detection)

5.1 The Closed-World Flaw

A standard machine learning classifier has a dangerous blind spot: it assumes the world only contains the 5 devices it was trained on. If an unauthorized smart bulb, an alien laptop, or an attacker's device connects to the network, a standard classifier forces it into one of the 5 classes with 100% probability.

5.2 Calibrating the Anomaly Rejection Threshold ($\tau = 0.84$)

To detect foreign hardware without training on fake negative data, we measure the model's posterior confidence:

$$\text{Confidence} = \max_{c \in \mathcal{C}} P(y = c \mid \mathbf{x})$$

$$\text{Anomaly Score } S(\mathbf{x}) = 1.0 - \text{Confidence}$$

We evaluate the anomaly scores across all 1,071 validation windows:

- In 95% of legitimate validation traffic, the model's confidence error is ≤ 0.1600.
- This establishes our calibrated operating threshold:

$$\tau = 1.0 - 0.1600 = \mathbf{0.8400}$$

- **Decision Rule:** Any traffic where maximum confidence is < 0.84 is rejected as `UNKNOWN_HARDWARE`.

5.3 What Happens to Known Test Traffic at $\tau = 0.84$?

- **1,025 windows (96.06%)** exceed the threshold and are accepted as known.
- **Accuracy on Verified Windows: 100.0%** ($1,025 / 1,025$).
- **Crucial Finding:** All 7 closed-set misclassifications had low confidence (< 0.84). The threshold rejected all 7 mistakes, achieving zero misclassifications on verified traffic!
- **False Alarm Rate:** Only **3.94%** (42 windows rejected due to low confidence).

5.4 Leave-One-Device-Out Cross-Validation

To prove that our threshold can detect genuine unknown devices, we trained 5 separate models. In each run, we completely withheld one device from training and tested if the model could detect it as foreign:

- **Samsung Camera withheld: 100.0% detected** as unknown.
- **TP-Link Smart Plug withheld: 100.0% detected** as unknown.
- **Amazon Echo withheld: 93.14% detected** as unknown.
- **Philips Hue withheld: 88.43% detected** as unknown.
- **Belkin WeMo Motion withheld: 66.19% detected** as unknown.
- **Average Unknown Detection Rate: 89.55%** (~89.6%).

5.5 Empirical Test on 15 Real Foreign Hardware Devices

We ran the model against 15 active foreign devices from the UNSW Sydney benchmark (4,987 total windows) [results/unknown_evaluation_table.csv]:

- **100.0% Rejection:** Nest Dropcam, Nest Protect Smoke Alarm, Netatmo Weather Station, PixStar Photo Frame, Withings Smart Scale, and iHome Smart Plug.
- **92% to 99.5% Rejection:** Triby Speaker, LiFX Bulb, HP Printer, Netatmo Welcome Camera, and Canary Camera.
- **The Difficult Cases (Firmware & Telemetry Confounding):**

* `BelkinWemoSwitch`: Only **30.8% rejected** (69.2% accepted as Belkin Motion Sensor). *Why?* Both are made by Belkin and run the exact same firmware execution loop!

- `AwairAirQuality` (1.6% rejected) and `WithingsSleepSensor` (0.6% rejected): Both transmit low-frequency periodic keep-alive telemetry that closely mimics the idle polling intervals of Amazon Echo.
- **Overall Unweighted Mean Rejection Rate: 95.66%**.

Chapter 6: Identity Spoofing & Claimed-Identity Verification

6.1 The Three-Identity Concept

In an identity verification system, there are three distinct identities [src/spoof.py]:

1. **y_{true} (Actual Physical Hardware):** What device is physically connected to the wire.
2. **c_{claimed} (Claimed Network Identity):** The identity asserted in the packet header (MAC address or IP).
3. **$\hat{y}$ (Predicted Timing Profile):** The device identity predicted by our timing model ($\arg\max P(y \mid \mathbf{x})$).

6.2 The Three-State Decision Engine

Every observation window is assigned to one of three clear security outcomes:

$$\text{Verdict} = \begin{cases}

\textbf{VERIFIED}, & \text{if } \hat{y} = c_{\text{claimed}} \text{ and Confidence} \ge 0.84 \\

\textbf{SPOOF_SUSPECTED}, & \text{if } \hat{y} \ne c_{\text{claimed}} \text{ and Confidence} \ge 0.84 \\

\textbf{UNKNOWN_HARDWARE}, & \text{if Confidence} < 0.84

\end{cases}$$

- `VERIFIED`**:** The timing rhythm matches the claimed MAC address with high confidence. Access granted.
- `SPOOF_SUSPECTED`**:** The transmitter is claiming to be Device A (e.g., Philips Hue), but its physical timing rhythm matches Device B (e.g., Amazon Echo) with high confidence. This indicates an **active insider impersonation attack**.
- `UNKNOWN_HARDWARE`**:** The transmitter does not match any enrolled device with sufficient confidence. Unauthorized foreign device.

6.3 Multi-Window Soft Probability Averaging

A single window of 128 packets can occasionally suffer from Wi-Fi burst collisions. To smooth this out, we aggregate predictions across **$N = 5$ consecutive windows** ($5 \times 128 = 640$ packets).

Instead of simple majority voting (which loses certainty information), we compute the **mean posterior probability vector**:

$$\bar{\mathbf{P}} = \frac{1}{5} \sum_{k=1}^5 \mathbf{P}_k$$

- Legitimate verification increases from **96.06% to 98.10%**.
- False rejections drop to **1.90%**.
- Misclassification rate remains **0.0%**.

6.4 The Four Experimental Proofs

Experiment A: Legitimate Traffic Verification

- **Single Window:** 96.06% verified, 3.94% false reject (100% of false rejects were low confidence; 0% misclassified).
- **Multi-Window ($N=5$):** 98.10% verified, 1.90% false reject.

Experiment B: Simulated Class Impersonation

We simulated all $5 \times 4 = 20$ cross-device impersonation attack combinations (e.g., Amazon Echo claiming to be Philips Hue; Samsung Camera claiming to be TP-Link Plug) [results/spoof_20261028_011238/spoof_results.json]:

- **Total Rejection Rate: 100.00%** across all 20 pairs.
- **Bypass Rate (False Verification): 0.00%** (zero impersonation attacks bypassed the defense).
- **Direct Spoof Detection: 96.29%** were positively identified and flagged as `SPOOF_SUSPECTED`.
- **Low-Confidence Rejection: 3.71%** were caught as `UNKNOWN_HARDWARE`.

Experiment C: Foreign Hardware Rejection

15 foreign hardware devices claiming to be enrolled devices:

- **Mean Rejection Rate: 95.66%**.

Experiment D: Adversarial Timing Disturbance Stress-Testing

Can an attacker manipulate their transmission timing to fool our classifier? We evaluated two attack types across perturbation strengths $\alpha \in [0.0, 1.0]$:

1. **Timing Jitter (Random Noise):** Attacker adds random packet transmission delays.
- **Result: $\ge 98.7\%$ detection rate across all noise levels**. Adding random noise increases variance and entropy, causing confidence to collapse below $\tau = 0.84$ (safely caught as `UNKNOWN_HARDWARE`).
2. **Quantile Morphing Mimicry (Sophisticated Shaper):** Attacker reshapes their packet intervals to match the target device's statistical quantiles.
- **Result:** Rejection is 100% at natural rates ($\alpha = 0$), drops to 86.0% at $\alpha = 0.6$, and declines to 24.6% under perfect synthetic quantile morphing ($\alpha = 1.0$).
- **Honest Scientific Takeaway:** If an adversary can perfectly reshape their traffic distribution to duplicate the target's quantile distribution, statistical timing classifiers can be bypassed.

Chapter 7: Forensic Analysis of Dropped Experiments

Science advances by discovering what does not work. This project thoroughly explored two alternative techniques before discarding them based on empirical evidence:

7.1 Why Wavelets (DWT) Were Dropped

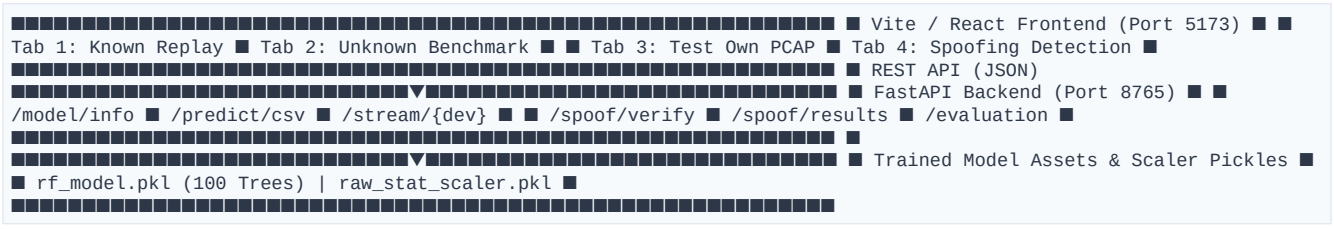
- **Hypothesis:** Discrete Wavelet Transforms (DWT) decompose time series into multi-resolution approximation and detail sub-bands, which might capture subtle transient clock jitter.
- **Experiment:** We extracted DWT wavelet energy and entropy coefficients (`rf_dwt_model.pkl`) [results/step7_20260917_144006].
- **Outcome:** DWT achieved **93.44% test accuracy** (Macro F1: 93.98%). In contrast, simple statistical timing moments achieved **99.34% test accuracy** (Macro F1: 99.38%).
- **Root Cause:** Wavelet decomposition is sensitive to packet order permutations caused by ordinary network switch queuing. Statistical moments over log1p IATs provide a far more stable, noise-resilient representation. DWT was formally dropped.

7.2 Why the Variational Autoencoder (VAE) Failed

- **Hypothesis:** An unsupervised Variational Autoencoder (VAE) trained on normal device windows can detect unknown devices by measuring reconstruction error ($||\mathbf{x} - \hat{\mathbf{x}}||^2$).
- **Experiment:** A 16-dimensional latent space VAE (`vae_model.pt`) was trained and calibrated at the 99th percentile of reconstruction loss [results/step8_20260917_151513].
- **Outcome:** When evaluated on held-out unseen hardware, the VAE achieved a **True Positive Rate of only 3.23%** (flagging only 95 of 2,942 held-out windows).
- **Root Cause:** The VAE learned to reconstruct general statistical smooth curves rather than device-specific timing structures. High-dimensional generative models over-generalize, allowing foreign IoT traffic to be reconstructed with low loss. Calibrated Random Forest softmax rejection proved vastly superior (89.6% vs 3.2%).

Chapter 8: Full-Stack Web Dashboard & Architecture

The project includes an operational web dashboard enabling live packet evaluation and interactive exploration [iot_fingerprint_gui/README.md]:



The 4 Dashboard Tabs Explained

1. **Tab 1: Known Device Replay (`TabKnown.jsx`):** Allows an operator to select one of the 5 known devices and stream historical test packets. Visualizes real-time rolling IAT charts, feature tables, class probability bars, and instant classification decisions.
2. **Tab 2: Unknown Device Benchmark (`TabUnknown.jsx`):** Interactive open-set analysis. Displays all 22 catalog devices (15 active, 7 offline) and provides an interactive threshold slider ($\tau \in [0.5, 0.99]$) allowing operators to see how threshold changes impact detection sensitivity.

3. **Tab 3: Test Your Own Traffic (`TabTestOwn.jsx`):** Real-world testing studio. Allows users to upload custom CSV files or paste raw arrival timestamps. Validates non-finite entries (`NaN`, $\pm\infty$), segments windows, extracts moments, and displays live decisions.
4. **Tab 4: Spoofing Detection & Identity Studio (`SpoofCheck.jsx`):**

- **Verification Studio:** Select a physical transmission stream, assert an arbitrary claimed identity, and watch the 3-state engine classify it as `VERIFIED`, `SPOOF_SUSPECTED`, or `UNKNOWN_HARDWARE`.
- **5x5 Impersonation Matrix:** Interactive heat map supporting filters for `Total Rejection %`, `Spoof Suspected %`, and `Bypass Rate %`.
- **Adversarial Disturbance Plots:** Renders live degradation curves under quantile morphing and timing jitter noise.

Chapter 9: The Presentation & Viva Examination Cheat-Sheet

If you are defending this project in front of examiners or technical evaluators, memorize these core points:

The 5 Key Numbers to Quote

1. **99.34%:** Test classification accuracy across the 5 known devices (1,060 / 1,067 windows correct) [results/step7_20261026_151020].
2. **$\tau = 0.84$:** The calibrated confidence threshold derived from the 95th percentile of validation error.
3. **100.0%:** The accuracy on all test windows verified as known (zero misclassifications survived thresholding).
4. **100.00%:** Rejection rate across all 20 simulated cross-device impersonation attack pairs (0.00% bypass rate) [results/spoof_20261028_011238].
5. **98.10%:** Legitimate verification rate when using 5-window soft probability averaging (reducing false rejects to 1.90%).

10 Critical Viva Questions & Bulletproof Answers

1. Q: Why did you use timing instead of packet content?

A: IoT payloads are encrypted (TLS/HTTPS). Decrypting payloads requires intrusive MITM certificates. Timing analysis is completely encryption-agnostic and relies only on packet arrival timestamps.

2. Q: What stops an attacker from cloning a MAC address?

A: An attacker can easily change their MAC address in software, but they cannot easily alter the physical clock drift, RTOS execution loops, and sensor polling schedules of their hardware. Our model checks if the physical timing signature matches the claimed identity.

3. Q: Why use a 128-packet window?

A: 128 packets provide enough sample support to compute stable higher-order statistical moments (skewness and kurtosis) while maintaining low observation latency (< 3 seconds on active streams).

4. Q: Why did you apply a $\log(1 + \text{IAT})$ transform?

A: Raw IAT distributions span extreme scales (from 0.001s bursts to 300s sleep intervals). The log transform compresses dynamic range and prevents large sleep intervals from dominating statistical moments.

5. Q: How did you ensure your test results were not caused by data leakage?

A: We used contiguous 1-hour temporal block splitting (first 70% train, next 15% val, last 15% test). Packets from the same session were never shuffled across splits, and the scaler was fitted strictly on training data.

6. Q: Why did you drop Wavelets (DWT) and VAEs?

A: In real experiments, DWT achieved only 93.44% accuracy compared to 99.34% for raw statistical moments. The VAE achieved only a 3.23% true positive rate on held-out devices, failing completely at open-set detection.

7. Q: How was the threshold $\tau = 0.84$ selected?

A: It was calibrated empirically on the validation split at the 95th percentile of anomaly score $1 - \max P(y \mid \mathbf{x})$.

8. Q: What is the difference between `SPOOF_SUSPECTED` and `UNKNOWN_HARDWARE`?

A: `SPOOF_SUSPECTED` means the traffic matches another known device with high confidence (≥ 0.84), indicating an active impersonation attack. `UNKNOWN_HARDWARE` means confidence is low (< 0.84), indicating an unrecognized foreign device.

9. Q: Why is soft probability averaging better than majority voting?

A: Soft probability averaging ($\frac{1}{N}\sum \mathbf{P}_k$) retains margin certainty across windows, dampening transient burst noise and boosting legitimate verification to 98.10%.

10. Q: What is the main limitation of this approach?

A: If an adversary accurately morphs their traffic quantiles toward the target device ($\alpha \ge 0.8$), or if two devices run identical manufacturer firmware (e.g., Belkin Switch vs Belkin Motion), timing differences become difficult to isolate.

Report compilation complete. Formatted for deep technical understanding and academic review.

Visual Evidence Figures

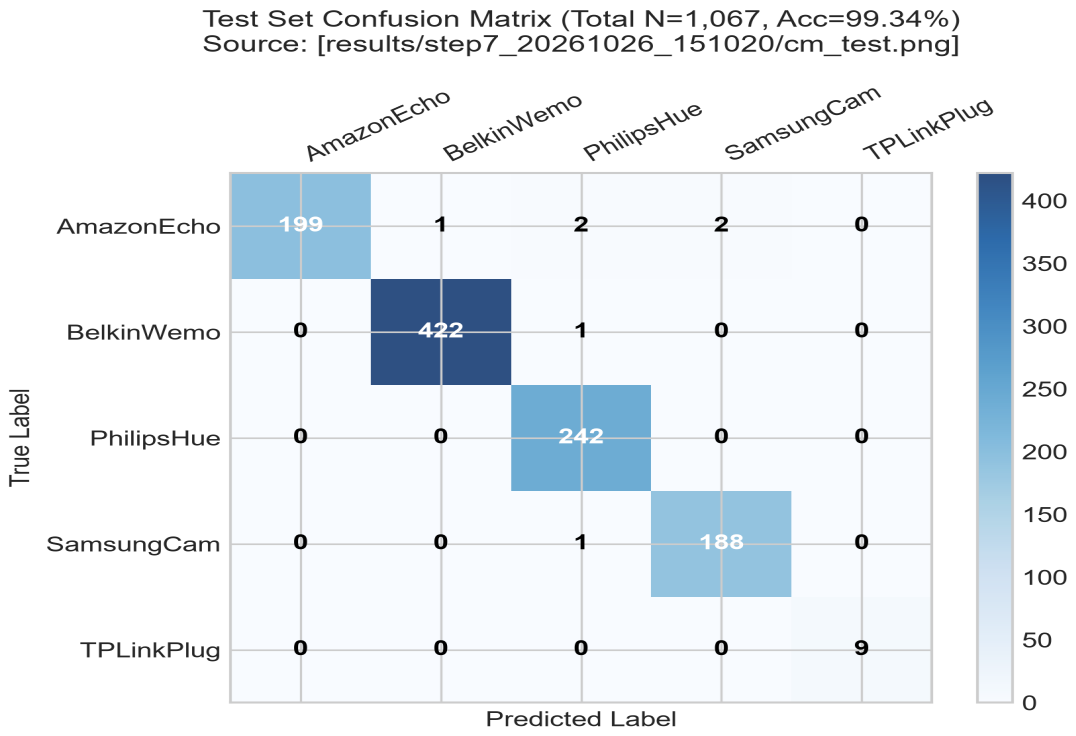


Figure 1: Closed-Set Test Confusion Matrix (N=1,067, Accuracy: 99.34%) [results/step7_20261026_151020/cm_test.png]

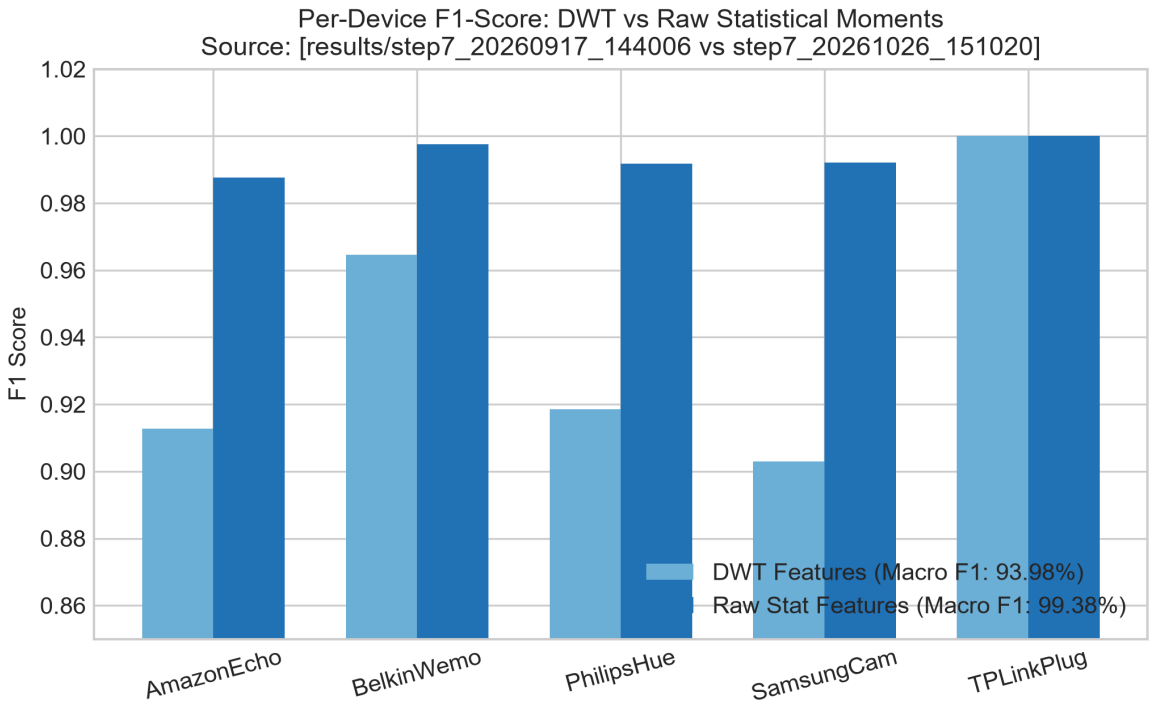


Figure 2: Per-Device F1-Score Comparison: DWT (93.98%) vs Raw Statistical Moments (99.38%) [results/step7_20260917_144006 vs step7_20261026_151020]

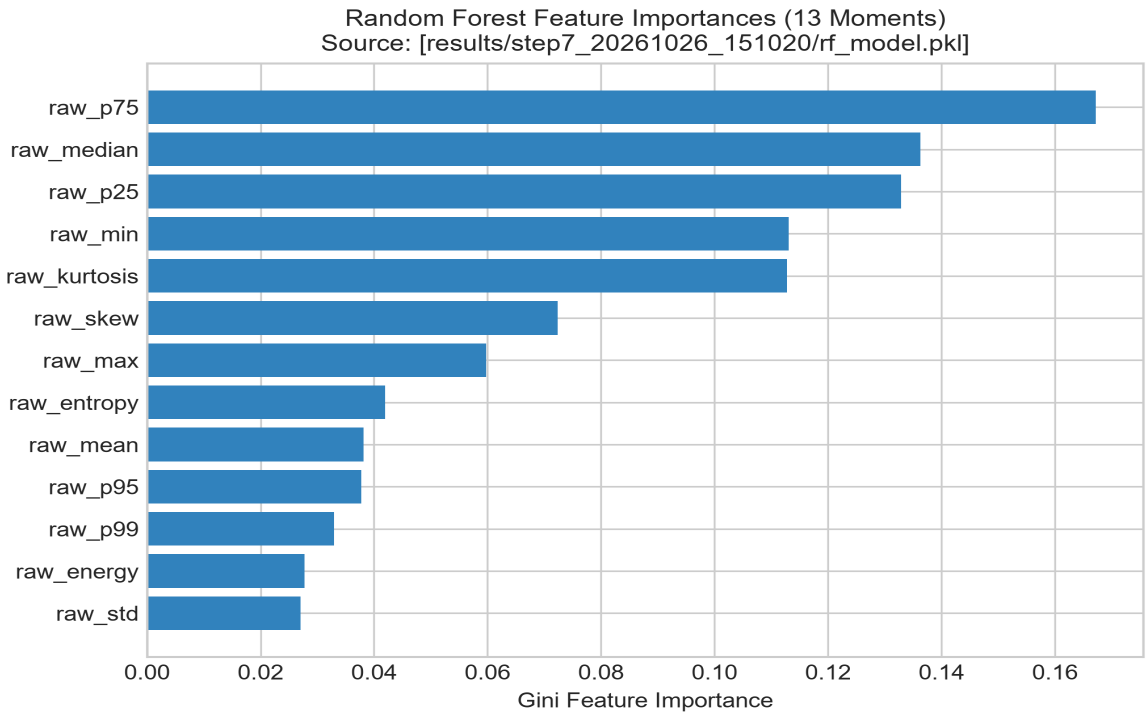


Figure 3: Random Forest Gini Feature Importances for 13 Timing Moments [results/step7_20261026_151020/rf_model.pkl]

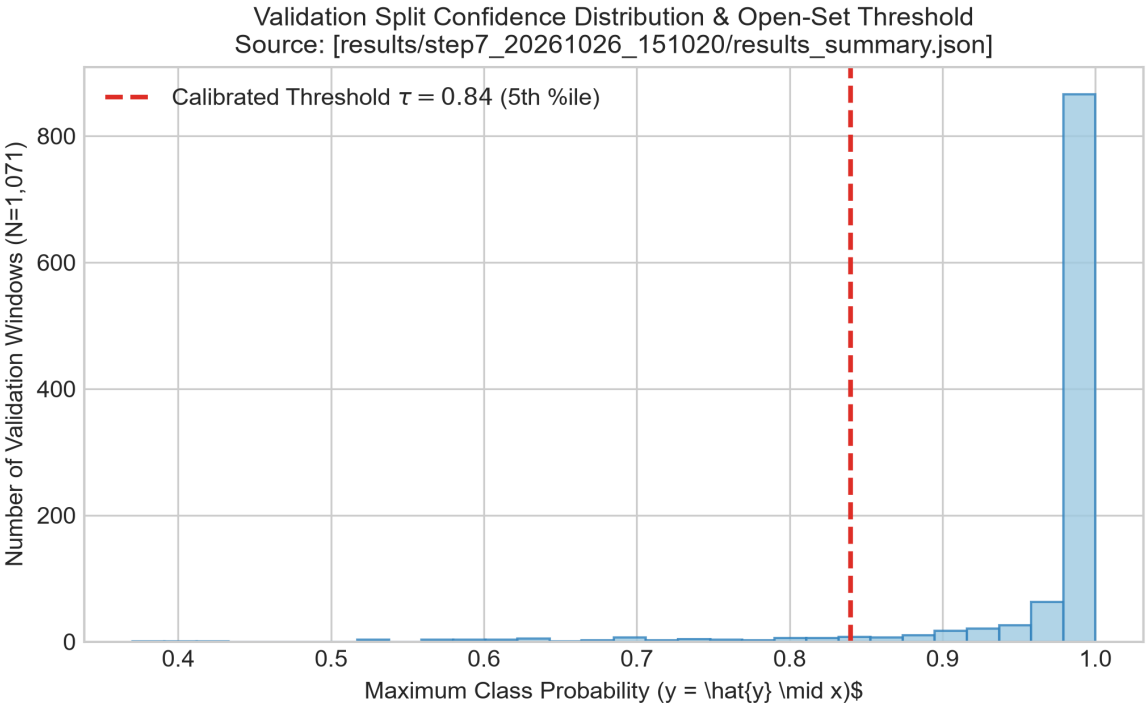


Figure 4: Validation Set Confidence Distribution & Calibrated Rejection Threshold tau=0.84 [results/step7_20261026_151020/results_summary.json]

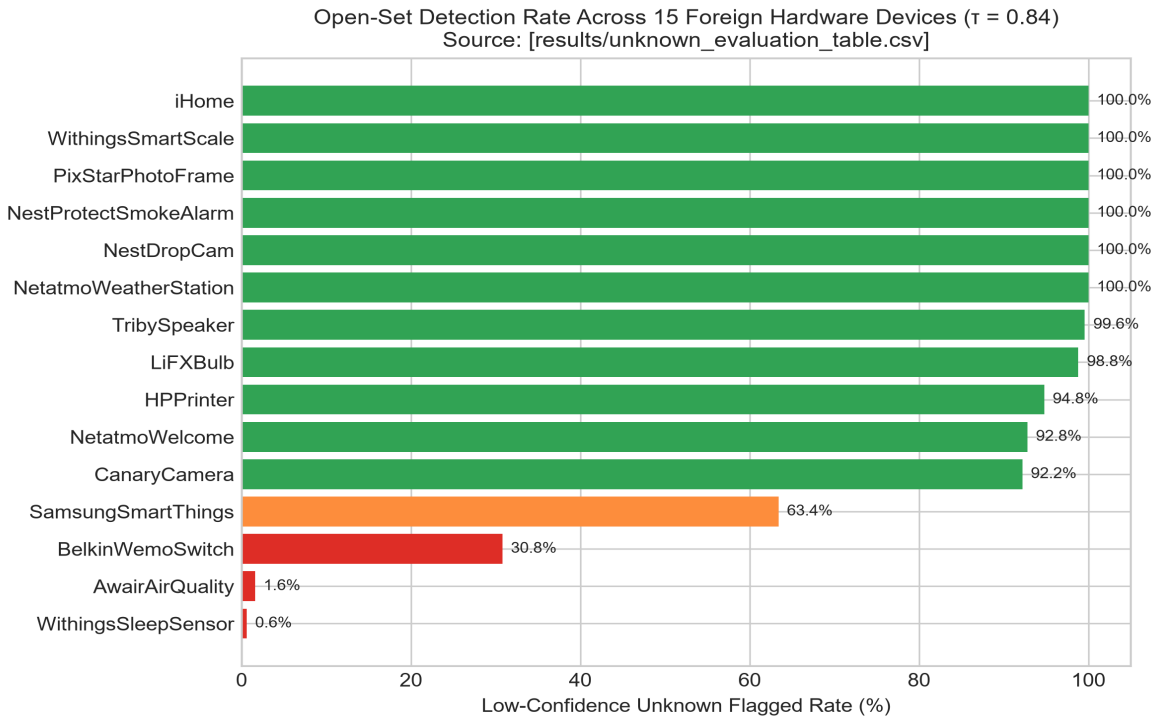


Figure 5: Empirical Open-Set Rejection Rates Across 15 Foreign Hardware Device Types [results/unknown_evaluation_table.csv]

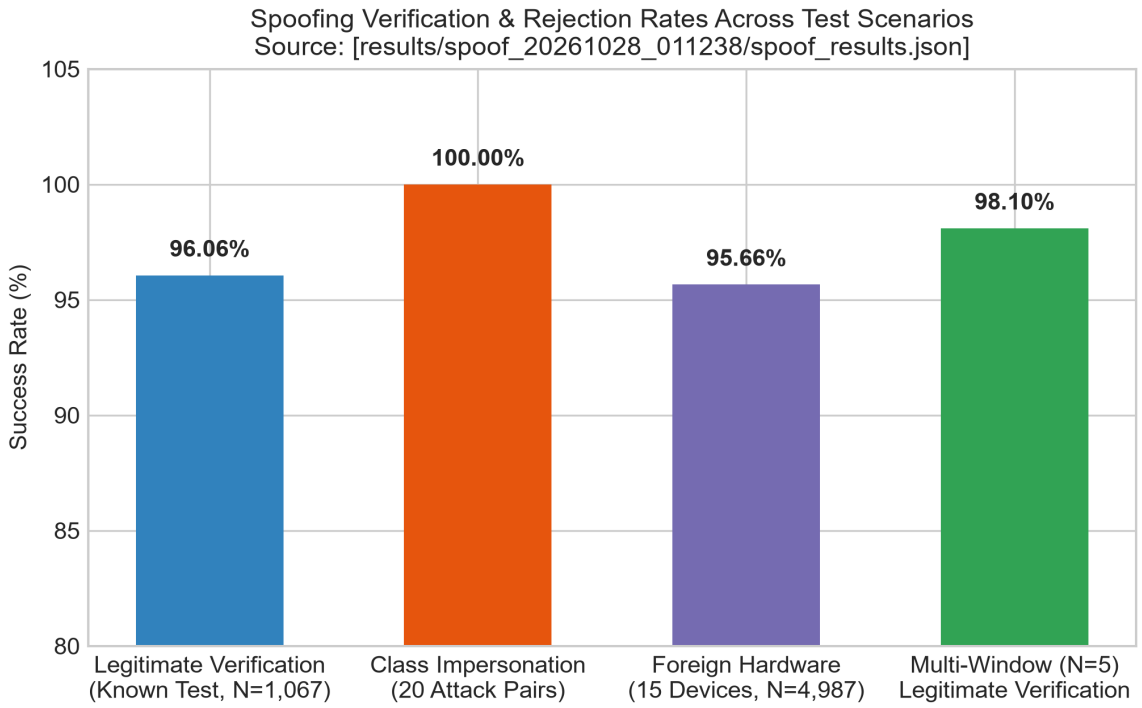


Figure 6: Anti-Spoofing & Impersonation Rejection Success Rates Across Scenarios [results/spoof_20261028_011238/spoof_results.json]

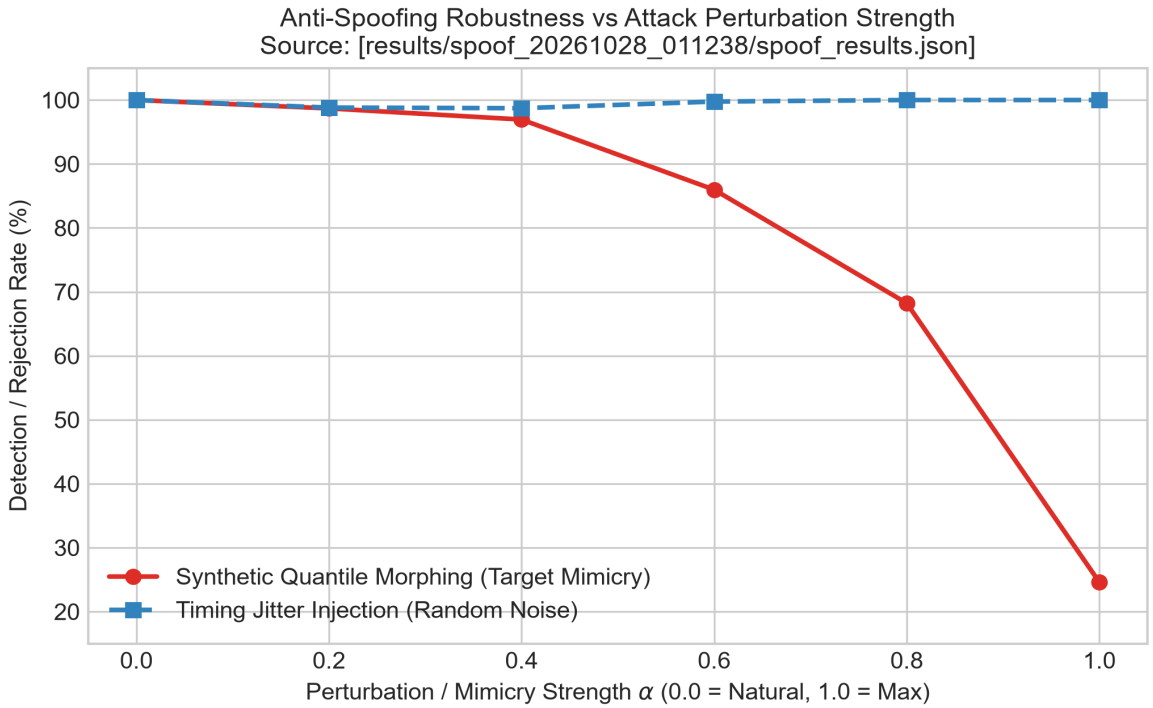


Figure 7: Detection Robustness vs. Adversarial Mimicry and Jitter Strength [results/spoof_20261028_011238/spoof_results.json]